

Requirements for the Development of a Website Builder with Adaptive Design

Gulmira Bekmanova

*L.N. Gumilyov
Eurasian National University
Astana, Kazakhstan*

Banu Yergesh

*Digital development and online
learning department
L.N. Gumilyov
Eurasian National University
Astana, Kazakhstan*

Assel Omarbekova

*Digital development and online
learning department
L.N. Gumilyov
Eurasian National University
Astana, Kazakhstan*

Laura Orynbay

*Faculty of Information Technologies
L.N. Gumilyov
Eurasian National University
Astana, Kazakhstan*

Aiganym Bessembayeva

*Digital development and online
learning department
L.N. Gumilyov
Eurasian National University
Astana, Kazakhstan*

Dinara Kabylova

*Digital development and online
learning department
L.N. Gumilyov
Eurasian National University
Astana, Kazakhstan*

Altanbek Zulkhazhav

*Digital development and online learning department
L.N. Gumilyov Eurasian National University
Astana, Kazakhstan*

Beibarys Sultan

*Digital development and online learning department
L.N. Gumilyov Eurasian National University
Astana, Kazakhstan*

Abstract—This article presents the requirements for developing a system that enables users with limited technical knowledge to create responsive websites effectively. The project, funded by the Ministry of Science and Higher Education of the Republic of Kazakhstan, seeks to combine automated website generation with adaptive design principles to produce user-friendly and accessible web resources. A comprehensive review of existing approaches to automatic website creation has been conducted, categorizing technologies based on layout, example-based methods, and artificial intelligence. The evolution of adaptive website design is discussed, highlighting the need for contemporary content-oriented and device-responsive web developments. The integration of server-side rendering (SSR) with Nuxt.js has been identified as a significant improvement in web development. This approach provides several benefits, including increased productivity, improved search engine optimization (SEO), and the ability to generate content dynamically. The article discusses the technical aspects of implementing SSR with Nuxt.js, including developing a prototype system with features such as a user-friendly website builder, automated content display, and responsive design tailored to different devices and users' needs. The prototype system demonstrates the potential for commercialization and the creation of accessible solutions for users with visual impairments. This work represents an important step towards making web development more inclusive, allowing users without advanced technical skills to create responsive websites. By using SSR and Nuxt.js, developers can create more accessible and user-friendly web experiences for a wider audience.

Keywords—*Automatic Website Generation, Adaptive Web Design, Server-Side Rendering (SSR), Nuxt.js*

This research is funded by the Scientific Committee of the Ministry of Science and Higher Education of the Republic of Kazakhstan (Grant No. AP23489504)

I. INTRODUCTION

The increasing demand for accessible and user-friendly web development tools has led to the creation of systems that allow individuals with minimal technical expertise to create and manage websites efficiently. This paper presents an advanced approach to website development using Nuxt.js for server-side rendering (SSR) combined with adaptive design principles. Unlike existing systems, our approach offers a modular architecture that simplifies the development process while improving the performance and adaptability of the website. This research contributes to this field by presenting a unique combination of SSR and adaptive design aimed at non-technical users to bridge the gap between usability and technical complexity in website development.

This article discusses the requirements for developing a system that allows users without deep knowledge of web development to create websites with adaptive design. It includes functions for automatically displaying created content on official websites, making it easy to manage and update web resources. Special attention is paid to integrating with adaptive designs that adapt to the features of user content and devices.

The article focuses on the creation of a technology for automated website development with adaptive design within a project funded by the Ministry of Science and Higher Education of the Republic of Kazakhstan. The research is being carried out within the framework of grant no. AP23489504. The practical significance of these results lies in their potential commercialization as separate products, services, and plugins for creating websites that are accessible to people with visual impairments. Modern requirements for web development set the task of creating accessible and user-friendly systems for a

wide range of users. With the increasing need for adaptive and inclusive web resources, there is a need to develop tools that allow users without special knowledge to create and manage such resources.

The system to be developed will allow users to create various website elements such as blogs and plugins without in-depth web development knowledge and display the created content automatically. The article gives an overview of existing methods for automatic website generation, examines the development of adaptive website design and highlights the integration of SSR with Nuxt.js to optimise dynamic content and adaptive functions. Finally, it presents the system requirements and a prototype that demonstrates the practical application of these technologies in the development of a user-friendly, adaptive website builder.

II. LITERATURE REVIEW

A. Automatic Website Generation

To lay the foundation for our system's development, it's essential to understand the current landscape of automatic website generation. Automatic website generation uses software tools and platforms to allow users to create a website without writing code. These tools provide templates, a drag-and-drop editor, and artificial intelligence features to simplify the website development process, making it easier for anyone, even those with limited technical knowledge. A systematic review of the literature conducted by [1] provides a comprehensive analysis of various approaches and techniques in the field of automatic website generation. The review divides automatic website generation into three main categories: mock-up-driven, example-based, and artificial intelligence-driven approaches. By directly converting a wireframe sketch or mock-up design of a website into functioning Graphical User Interface (GUI) code, the mock-up-driven automatic website creation approach creates a website prototype automatically [2] [3]. Additionally, by using real-world websites as examples, an example-based method for automatic website generation creates bespoke websites automatically [4]. Lastly, by learning about users' needs, the AI-driven website-generating approach automatically creates user-preferred websites [5].

B. Evolution and Implementation of Adaptive Website Design

While the automatic generation of websites forms the basis for web content, ensuring adaptability to user needs is crucial. The evolution of web design, characterised by the emergence of adaptive and responsive approaches, has brought about a change. Originally, websites used fixed layouts that were optimised for a limited number of devices. However, the growing variety of devices with different screen sizes required more flexible solutions. This led to the development of Responsive Web Design (RWD) by Ethan Marcotte in 2010, which introduced a fluid grid system for different devices. In parallel, Adaptive Web Design (AWD) used predefined layouts for specific devices that were optimised for specific screen sizes and functions [6].

The self-adapting adaptive website design, a further development of RWD, takes the complexities of different modern devices into account. Traditional web design required multiple versions for different devices, which was very resource-intensive. RWD offered a solution by allowing websites to automatically adapt layout and content based on device and

screen size. Recent advancements have introduced content-based customisation so that websites can also adapt to the type of content. For example, multimedia websites can prioritise the display of videos on larger screens, while text is optimised for readability on smaller devices [7].

Implementing such adaptive designs often involves the use of CSS media queries, JavaScript, and other web technologies that detect the user's environment and apply appropriate styling rules. The concept of Responsive Universal Design (RUD) extends this by incorporating user profiles and preferences into the adaptive process, ensuring that individual user needs, such as visual impairments or mobility challenges, are considered alongside device specifications. Moreover, frameworks like React JS have further enhanced the development of sophisticated adaptive websites. React's component-based architecture allows for modular and reusable components that can be dynamically rendered based on the user's context, facilitating seamless adaptation of layouts across different devices and content types [8].

In educational settings, responsive design has been particularly impactful, transforming e-learning platforms to accommodate the diverse range of devices used by students and educators [9]. By applying responsive design principles, educational websites can ensure that learning materials remain accessible and user-friendly, regardless of whether they are accessed on a desktop or a mobile device, thereby significantly enhancing the overall learning experience [10].

The self-adapting adaptive website design reorganises and reformats content to adapt it to the respective viewing context. For example, images can be displayed in a grid on desktops, while they are displayed in a single-column format on mobile devices. For text-heavy pages, the font size, line spacing and contrast are adjusted to improve readability based on the display characteristics of the device. This adaptability is driven by a rule-based approach such as Xadaptor, which uses predefined rules and fuzzy logic to ensure consistent display quality across devices by categorising HTML objects [11].

Additionally, frameworks for content adaptation based on co-browsing tasks have emerged, which consider both the display capabilities of each device and the shared and personal viewpoints of users engaged in co-browsing sessions. This ensures that all participants, regardless of their device, can meaningfully interact with the content, as the framework dynamically adapts the content layout to suit the smallest device's display while offering additional views for larger screens [12].

Self-adapting adaptive website design represents a significant leap in web development by combining responsive design, content-based customisation and user-centric approaches such as RUD. This enables developers to create websites that provide a consistent and optimised user experience across different devices and contexts. By integrating rule-based systems and adaptive frameworks, websites can intelligently adapt their layout and presentation to the content and context, improving usability and accessibility.

Adaptive design has become an important feature of modern web development, especially on platforms that aim to create accessible, adaptive and user-friendly websites for a variety of devices. Several website builders such as Weebly [13], Wix [14] and Ecwid [15] demonstrate effective application of adaptive design principles, making them a popular choice for both e-commerce and educational purposes.

Website builders such as Weebly, Wix and Ecwid offer powerful tools that make it easy to create websites that automatically adjust to screen size and device type. These platforms are designed with adaptive capabilities in mind, allowing users to create sites that remain functional and visually appealing on desktops, tablets and smartphones alike.

For example, Wix provides a ‘drag and drop’ interface where users can create sites that dynamically adjust elements such as navigation menus, product images and text blocks to fit the screen size. Similarly, Weebly offers a flexible environment for creating adaptive sites, ensuring that content is easily accessible and well-organised on smaller screens. Ecwid, primarily used for e-commerce, improves user experience by automatically scaling and rearranging features such as shopping baskets and catalogues, making them user-friendly across devices.

Integrating such adaptive design features into website builders not only improves usability but also ensures that websites are accessible to a wide audience, regardless of the device used. This adaptability is key to maintaining a unified user experience and meeting the diverse needs of users in different contexts [16].

In the context of education, adaptive design plays a crucial role in creating inclusive and effective learning environments. The use of website builders in educational projects such as the Website Builder-Based Project (WBP) for learning English for Specific Purposes (ESP) [17] demonstrates the importance of adaptive design in improving the educational process.

These educational platforms utilise website builders to create digital learning environments that adapt to different devices. For example, sites created with these tools automatically change layout and content based on screen size, ensuring that students can access materials and participate in collaborative activities from laptops, tablets or mobile phones alike. This adaptability is particularly useful in smart learning environments where students and teachers rely on a variety of devices to engage with digital content.

The adaptive design features of these website builders ensure that educational content is not only accessible but also engaging and interactive across all devices. This is critical to support collaboration and reflection in online learning environments where students may need to navigate, interact and participate in digital projects from multiple platforms [17].

C. Server-Side Rendering with Nuxt.js for Adaptive Websites

To further improve the performance and adaptability of our system, we integrate server-side rendering techniques. Server-side rendering (SSR) plays a crucial role in adaptive design. It involves rendering web pages on the server instead of in the browser, resulting in faster load times and improved search engine optimization (SEO) performance [18]. SSR is particularly beneficial for websites with a lot of content, as it allows for pre-rendering HTML pages, making them available to users without the need for extensive client-side processing [19]. This approach ensures that websites can adapt based on content while maintaining performance and responsiveness.

Several SSR frameworks have emerged as popular choices for developers looking to implement server-side rendering in their web applications. Some of these frameworks include Next.js, Angular Universal, and Sapper.

- Next.js is a widely adopted client-side rendering (CSR) framework, particularly in the React community. It allows

developers to create both static and dynamic web applications easily. Next.js supports hybrid rendering, combining SSR and static site generation (SSG), making it a great choice for projects that prioritize performance and SEO. It also has an intuitive file-based routing system and API routes, simplifying the development process [20].

- Angular Universal extends the capabilities of Angular by enabling SSR. This framework allows Angular applications to be rendered on the server, significantly improving the initial load times of the application and making it more search engine optimized (SEO-friendly). Angular Universal seamlessly integrates with existing Angular applications, utilizing Angular’s powerful dependency injection system and component architecture to provide a smooth transition to SSR [21].
- Sapper, built on top of Svelte, is an SSR framework that is known for its simplicity and performance. It allows developers to create highly optimized web applications with minimal configuration. While Sapper has been replaced by Svelte Kit, it remains an option for those who prefer working within the Svelte framework [21].

These SSR frameworks have unique strengths, and choosing which one to use depends on the specific needs of the project. But when it comes to creating self-adjusting, adaptive websites, Nuxt.js stands out as a compelling option thanks to its powerful features and flexibility.

Nuxt.js offers a robust framework for implementing SSR in adaptive web design, built on Vue.js. Its hybrid approach allows developers to easily switch between SSG and SSR based on content needs. This flexibility is crucial when working with websites that need to adapt dynamically to different types of content.

Nuxt.js’s modular architecture allows developers to break down complex applications into manageable modules, promoting code reuse and efficient content-based component adjustments. It supports dynamic component imports, leading to faster load times and efficient rendering. Additionally, Nuxt.js offers an advanced routing system that, combined with asynchronous data handling, ensures responsive layouts that adapt in real-time [21] [20].

Performance is another strength of Nuxt.js. A study conducted by Siahaan, M., & Kenidy, R. compared rendering performance among popular frameworks, including Nuxt.js, and found that Nuxt.js performs competitively, especially in scenarios involving complex content rendering [21]. The framework optimizes the loading process with features such as automatic code splitting and caching, ensuring websites remain fast and responsive as they adapt to changing content. Nuxt.js provides a built-in solution for managing metadata and the document head, which is essential for SEO optimization. This is particularly important for improving search engine visibility and ensuring that your website ranks well in search results [20].

Nuxt.js is an optimal choice for developing self-adapting, responsive websites that rely on SSR. Its modular architecture and advanced routing capabilities make it a powerful framework for creating dynamic content-driven experiences.

By using Nuxt.js, developers can create websites that meet the needs of modern users, while also maintaining high performance and SEO standards.

Fig. 1 and Fig. 2 demonstrate screenshots of the initial stages of the Nuxt.js implementation in website development.

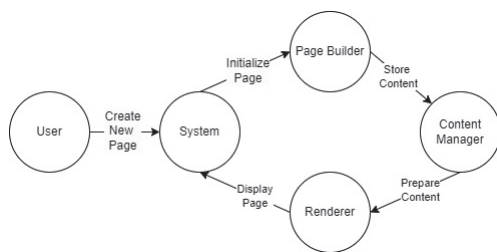


Fig. 4. Sequence Diagram for Page Creation.

elements onto a page. ContentManager manages the storage and retrieval of this data, and Renderer ensures that the content is displayed correctly on the front-end based on device type and screen size.

The system prototype is a simplified version of a website builder that allows users to create basic elements, such as pages and blogs. The created content is automatically displayed on a test website. We have also implemented a basic version of adaptive design that adapts to the screen size, but further optimization is required.

As part of our website development project, we actively used the capabilities of a website builder to manage menus and pages. Here are some examples of how we used various features to improve the structure and navigation of the site, as shown in Fig. 5 and Fig. 6.

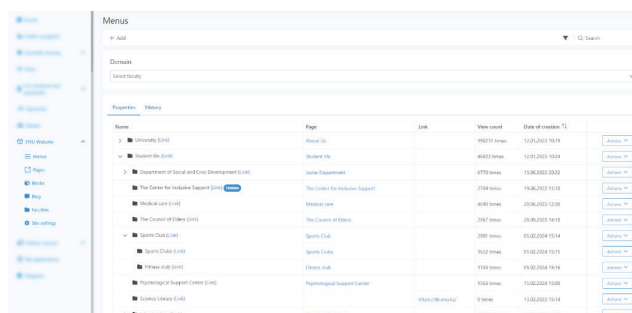


Fig. 5. A prototype for creating a menu in the website builder



Fig. 6. A prototype of the menu on the website

The website builder makes creating and editing pages easy, allowing users to fill them with content in multiple languages, as demonstrated in Fig. 7 and Fig. 8. Users can enter titles and content in Kazakh, Russian, and English when creating a new page. Once the fields are filled and the necessary files are downloaded, the page can be saved and made available on the site, simplifying the management of multilingual content and keeping the information up-to-date.

The website builder provides options for customizing the menu, including where it will be displayed on the main page

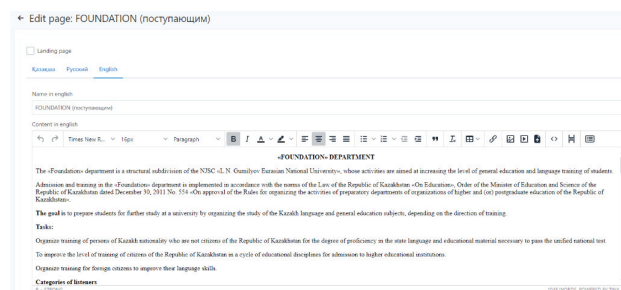


Fig. 7. A prototype for creating content in the website builder

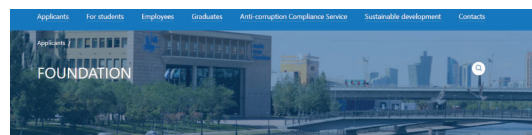


Fig. 8. A prototype of the content on the website

- in the header, in the middle, or hidden. Users also have the option to add useful links to a section, set the order of displayed elements, and upload background images. These features allow users to flexibly control their site's navigation and visual design as shown in Fig. 9 and Fig. 10.

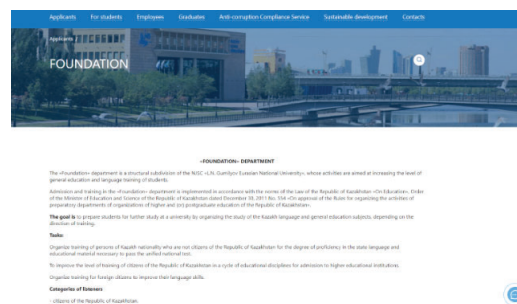


Fig. 9. A prototype for creating content in the website builder

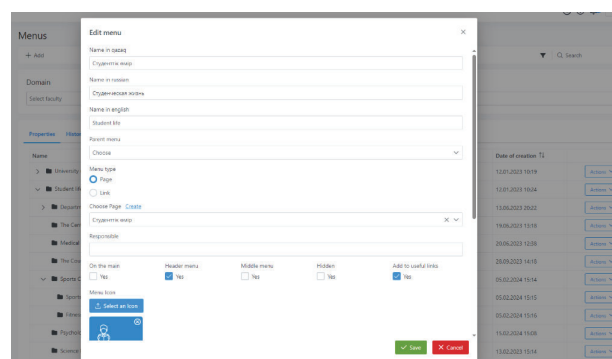


Fig. 10. A prototype of the content on the website

At the moment, the design and prototyping stages of the system have been completed. This will allow users with no knowledge of web development to create websites with adaptive design and automatic content display. The development is ongoing and in the near future, it is planned to fully implement all the planned features, making the system ready for practical use.

IV. DISCUSSION

Before the advent of adaptive technologies, websites were static and required manual updates to display new informa-

tion. This process was not only time-consuming but also error-prone, as each update had to be performed directly on the website by someone with sufficient technical skills. Managing content in such environments presented significant challenges, often resulting in outdated information and a lack of responsiveness to user needs. The introduction of adaptive technologies has revolutionised this workflow by allowing dynamic content to be automatically updated.

Such systems allow updates to be made within a centralised work platform where data is entered and website content is automatically updated in real time. This automation not only simplifies the website management process but also significantly reduces the need for constant manual intervention, minimising the risk of errors. As a result, the information displayed on the website remains consistently up-to-date, increasing reliability and user confidence.

The transition from a static to a dynamic, adaptive website significantly improves the user experience. Adaptive technology ensures that content is accessible and optimised across different devices such as smartphones, tablets and desktops. By automatically adapting to different screen sizes and orientations, adaptive websites provide a smooth and engaging browsing experience that increases user satisfaction and retention.

In addition, the move to dynamic, adaptive websites also impacts organisations by enhancing their ability to communicate effectively and reach a wider audience. As more users access content from mobile devices, it becomes important to ensure that interactions across platforms are consistent. This transition also supports mobile-first strategies, which are becoming increasingly important in today's digital environment.

The role of adaptive technologies in web development is only likely to increase in the future. As artificial intelligence and machine learning are increasingly integrated into content management systems, we can expect even more personalised and automated content delivery, further enhancing the adaptability and responsiveness of websites.

V. CONCLUSION

The development of an automated website generation system with adaptive design capabilities represents a significant step forward in making web development accessible to a broader audience, including those without technical expertise. This project, supported by the Ministry of Science and Higher Education of the Republic of Kazakhstan, has successfully combined various modern web technologies to create a prototype that not only simplifies the process of website creation but also ensures that the resulting sites are responsive and accessible across different devices.

Through a comprehensive review of current methodologies in automatic website generation and adaptive design, we identified the strengths and limitations of existing approaches. By integrating server-side rendering with Nuxt.js, our system leverages the benefits of modular architecture and advanced routing to enhance performance and adaptability, especially for content-rich websites.

The system development requirements laid out in this project emphasize the importance of a user-friendly interface, automated content management, and robust security measures, ensuring that the system is both practical and secure. The prototype developed so far demonstrates the core functionalities of the system, providing a solid foundation for further enhancements and commercialization.

As we move forward, the focus will be on refining the adaptive design features, expanding compatibility with existing content management systems, and enhancing the system's ability to cater to users with visual impairments. The ongoing development aims to deliver a fully functional and market-ready product that empowers users to create and manage websites that are both visually appealing and accessible, regardless of their technical background.

REFERENCES

- [1] T. Kaluarachchi and M. Wickramasinghe, "A systematic literature review on automatic website generation," *Journal of Computer Languages*, vol. 75, p. 101202, 2023.
- [2] M. W. Newman and J. A. Landay, "Sitemaps, storyboards, and specifications: A sketch of web site design practice," in *Proceedings of the 3rd conference on Designing interactive systems: processes, practices, methods, and techniques*, pp. 263–274, 2000.
- [3] F. K. Ozenc, M. Kim, J. Zimmerman, S. Oney, and B. Myers, "How to support designers in getting hold of the immaterial material of software," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pp. 2513–2522, 2010.
- [4] D. Gibson, K. Punera, and A. Tomkins, "The volume and evolution of web page templates," in *Special interest tracks and posters of the 14th international conference on World Wide Web*, pp. 830–839, 2005.
- [5] D. Ericsson and V. Byström, "Is the ai-generated web for everyone?: Examining the accessibility of ai-generated code by generative code tools in transition from prototype to development," 2024.
- [6] D. Rathore and N. Singhal, "Web design dilemma: A comprehensive guide to adaptive and responsive design," in *2024 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, pp. 1–5, IEEE, 2024.
- [7] H.-W. Yang and H. Mei, "Responsive universal design with universal user profiles and css user queries," 2014.
- [8] R. Aruna, G. Dhandayuthabani, M. M. J. Sindhia, M. KrishnaSaiVignesh, et al., "Responsive portfolio website using react," in *2023 2nd International Conference on Futuristic Technologies (INCOFT)*, pp. 1–5, IEEE, 2023.
- [9] B. Gulmira, M. Gulmira, O. Assel, O. Aigerim, Z. Altanbek, and S. Beibarys, "Aspects of digital transformation of higher education in the republic of kazakhstan," in *International Conference on Computational Science and Its Applications*, pp. 111–123, Springer, 2024.
- [10] B. Zhu, "Responsive design: E-learning site transformation," in *2013 Fourth International Conference on Networking and Distributed Computing*, pp. 126–130, IEEE, 2013.
- [11] J. He, T. Gao, W. Hao, I.-L. Yen, and F. Bastani, "A flexible content adaptation system using a rule-based approach," *IEEE Transactions on Knowledge and Data Engineering*, vol. 19, no. 1, pp. 127–140, 2006.
- [12] H.-N. Chua, S. D. Scott, Y. Choi, and P. Blanchfield, "Web-page adaptation framework for pc & mobile device collaboration," in *19th International Conference on Advanced Information Networking and Applications (AINA'05) Volume 1 (AINA papers)*, vol. 2, pp. 727–732, IEEE, 2005.
- [13] Square, "Weebly," n.d. [Online; accessed 2024-08-25].
- [14] Wix.com, "Wix," 2006. [Online; accessed 2024-08-25].
- [15] Lightspeed, "Ecwid," n.d. [Online; accessed 2024-08-25].
- [16] A. H. Kusumo, "Has website design using website builder fulfilled usability aspects? a study case of three website builders," in *Proceedings of the 4th International Conference on Informatics, Technology and Engineering 2023 (InCITE 2023)*, pp. 545–557, Atlantis Highlights in Engineering, 2023.
- [17] H.-T. T. Nguyen, "Website builder as an assistive technology tool for reflection, collaboration and skills development in learning esp.," *Journal of Learning for Development*, vol. 11, no. 1, pp. 138–150, 2024.
- [18] T. Gamma and N. Gerasimenko, *Swisscom Design System-Server-Side-Rendering*. PhD thesis, OST-Ostscherwe Fachhochschule, 2024.
- [19] A. Meredova, "Comparison of server-side rendering capabilities of react and vue," 2023.
- [20] A. Nguyen, "Building an e-commerce website using next js, mantine, and strapi," 2022.
- [21] M. Siahaan and R. Kenidy, "Rendering performance comparison of react, vue, next, and nuxt," *Jurnal Mantik*, vol. 7, no. 3, pp. 1851–1860, 2023.